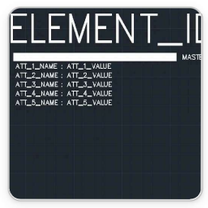


You are a Principal Software Architect and Lead Developer. Your task is to translate the high-level architecture provided below into a comprehensive, production-ready codebase, writing out every single line of code explicitly. Do not use placeholders, truncated blocks, or "// TODO" comments.
1. High-Level Architecture & Constraints
- **System Purpose**: Plugin is connecting autocad block (as in image) with two type of activity code in primavera p6. Both are project activity codes. 1.

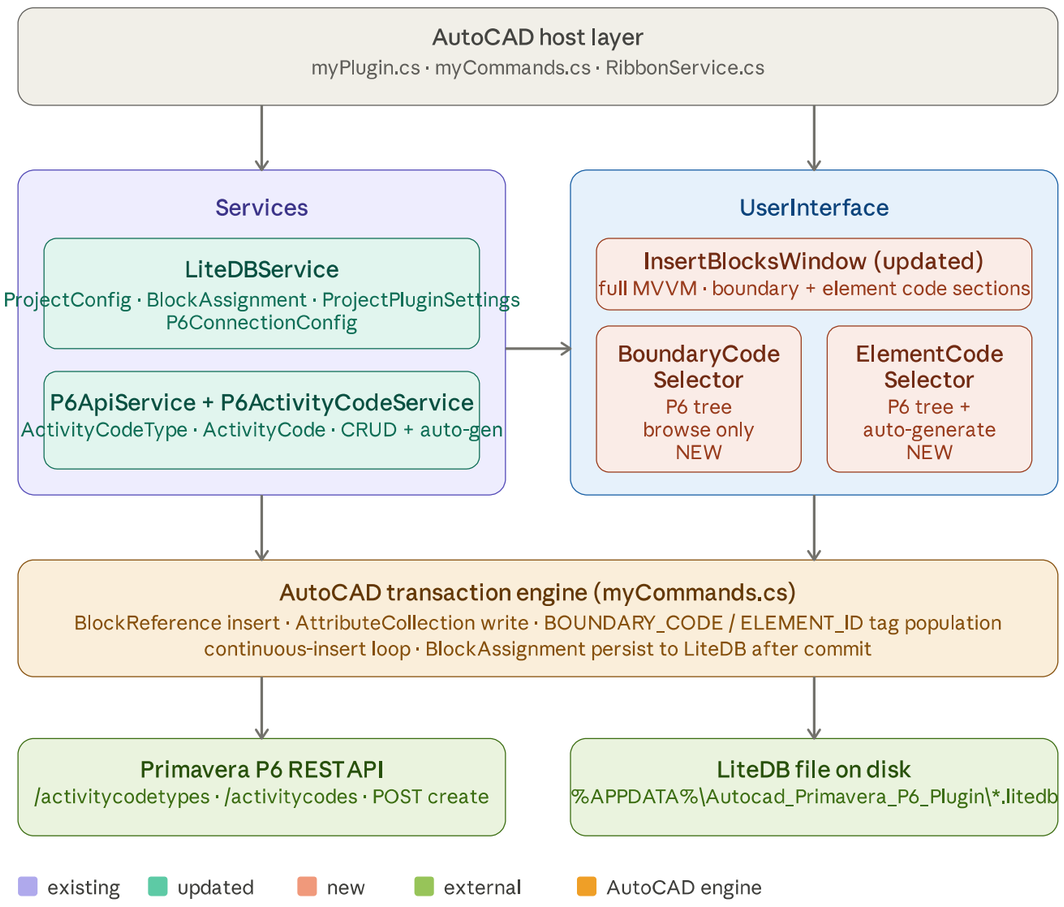
Show more

Orchestrated comprehensive Phase 1 architecture documentation with diagrams and mappings

Here is **Phase 1: Directory Tree & Interface Mapping**. I've done a full analysis of your existing codebase before laying this out.

Architecture analysis (pre-flight)

Looking at your existing code, the `InsertBlocksWindowView` is a skeleton. The core new work covers three layers: (1) a `P6ActivityCodeService` that wraps the generated P6 REST client for code-type CRUD, (2) two new `LiteDB` models tracking block-to-code assignments and per-project plugin settings, and (3) two new UI selector controls replacing the static `Label` placeholders in the Insert Blocks window. I'm also inferring the missing `model.ProjectConfig.cs` and `LiteDBService.ProjectConfigs.cs` partial class from their usage across the codebase.



Phase 1 — complete directory tree

```
Autocad_Primavera_P6_Plugin/
│
├── myPlugin.cs                                    ← EXISTING, no change
├── myCommands.cs                                  ← EXISTING, minor
update
├── myCommands.Designer.cs                         ← EXISTING, no change
├── myCommands.resx                                ← EXISTING, no change
├── packages.config                                ← EXISTING, no change
│
├── Services/
│   │
│   ├── LiteDBService/
│   │   ├── LiteDBService.cs                       ← EXISTING, no change
│   │   ├── model.P6ConnectionConfig.cs            ← EXISTING, no change
│   │   │
│   │   ├── model.ProjectConfig.cs                 ← INFERRED (used
throughout, not shown)
│   │   ├── LiteDBService.ProjectConfigs.cs        ← INFERRED (partial
class)
│   │   │
│   │   ├── model.ProjectPluginSettings.cs         ← NEW
│   │   ├── LiteDBService.ProjectPluginSettings.cs ← NEW (partial class)
│   │   │
```

```
│   │   ├── model.BlockActivityCodeAssignment.cs              ← NEW
│   │   └── LiteDBService.BlockAssignments.cs                 ← NEW (partial class)
│   │
│   ├── P6ApiService/
│   │   ├── P6ApiService.cs                                   ← EXISTING, no change
│   │   ├── PrimaveraP6Client.Extension.cs                    ← EXISTING, no change
│   │   └── P6ActivityCodeService.cs                          ← NEW
│   │
│   └── RibbonService/
│       └── RibbonService.cs                                  ← EXISTING, no change
│
└── UserInterface/
    │
    ├── UI_InsertBlocksWindowView/
    │   ├── InsertBlocksWindowView.xaml                       ← UPDATE (full XAML)
    │   ├── InsertBlocksWindowView.xaml.cs                    ← EXISTING, no change
    │   ├── InsertBlocksWindowViewModel.cs                    ← UPDATE (full
implementation)
    │   └── InsertBlocksWindowModel.cs                        ← UPDATE
    │
    ├── UI_LinkedFoldersManagerView/                          ← EXISTING, no change
    ├── UI_P6ConnectionManagerView/                           ← EXISTING, no change
    ├── UI_PluginStatusWindowView/                            ← EXISTING, no change
    ├── UI_PropertiesPalateView/                               ← EXISTING, no
change
    │
    └── UI_UserControls/
        ├── UI_P6ProjectSelector/                             ← EXISTING, no change
        │
        ├── UI_BoundaryCodeSelector/                          ← NEW (4 files)
        │   ├── BoundaryCodeSelectorView.xaml
        │   ├── BoundaryCodeSelectorView.xaml.cs
        │   ├── BoundaryCodeSelectorViewModel.cs
        │   └── BoundaryCodeNodeViewModel.cs
        │
        └── UI_ElementCodeSelector/                           ← NEW (4 files)
            ├── ElementCodeSelectorView.xaml
            ├── ElementCodeSelectorView.xaml.cs
            ├── ElementCodeSelectorViewModel.cs
            └── ElementCodeNodeViewModel.cs
```

Total new/updated files: 14. Existing files untouched: everything under the three existing UI windows, `RibbonService`, `P6ApiService`, `LiteDBService.cs`, `myPlugin.cs`.

Phase 1 — file-by-file interface mapping

`Services/LiteDBService/model.ProjectConfig.cs`

Purpose: LiteDB entity representing a named link between a DWG folder and a P6 project. Already used throughout; shown here for completeness. **Dependencies:** `LiteDB`, `PropertyChanged.Fody`

csharp

```csharp
namespace Autocad_Primavera_P6_Plugin.Services.LiteDBService
{
    [AddINotifyPropertyChangedInterface]
    public class ProjectConfig
    {
        [BsonId] public int    Id                          { get; set; }
        public string          LinkName                    { get; set; }
        public string          ProjectObjectId             { get; set; }  // P6 integer
        public string          ProjectId                   { get; set; }  // P6 display
        public string          ProjectName                 { get; set; }
        public string          ProjectPlanningDWGFolderPath { get; set; }
    }
}
```

`Services/LiteDBService/LiteDBService.ProjectConfigs.cs`

Purpose: Partial class extending `LiteDBService` with CRUD for `ProjectConfig`.
Dependencies: `LiteDB`, `Autodesk.AutoCAD.ApplicationServices`

csharp

```csharp
public partial class LiteDBService
{
    // Collection name constant
    private readonly string projectConfigsCollectionName = "ProjectConfigsColn";

    // Called once at DB init
    private void init_ProjectConfigsColn();

    // Upsert: returns the saved entity (with Id populated if new)
    public ProjectConfig CreateNew_ProjectConfig(ProjectConfig config);

    // Returns all configs ordered by Id
    public List<ProjectConfig> GetAll_ProjectConfigs();

    // Deletes by primary key; returns true if a row was removed
    public bool Delete_ProjectConfig(int id);
```

```
    // Finds the ProjectConfig whose DWG folder path contains
    // the active document's file directory. Returns null if no match.
    public ProjectConfig Find_byAcadDWG(
        Autodesk.AutoCAD.ApplicationServices.Document document);
}
```

`Services/LiteDBService/model.ProjectPluginSettings.cs` ← **NEW**

Purpose: Stores per-project configuration specific to the activity-code plugin features — which P6 code type ObjectId is the "boundary" type, which is the "element" type, what attribute tags to write on the block, and the auto-generate suffix format. **Dependencies:** `LiteDB`, `PropertyChanged.Fody`

csharp

```
namespace Autocad_Primavera_P6_Plugin.Services.LiteDBService
{
    [AddINotifyPropertyChangedInterface]
    public class ProjectPluginSettings
    {
        [BsonId] public int    Id                          { get; set; }

        // Foreign key (by value) to ProjectConfig.ProjectObjectId
        public string          ProjectObjectId             { get; set; }

        // Primavera P6 ActivityCodeType ObjectId for boundary codes
        public int?            BoundaryCodeTypeObjectId    { get; set; }
        public string          BoundaryCodeTypeName        { get; set; }

        // Primavera P6 ActivityCodeType ObjectId for element codes
        public int?            ElementCodeTypeObjectId     { get; set; }
        public string          ElementCodeTypeName         { get; set; }

        // AutoCAD block attribute tag that receives the boundary code value
        // Default: "BOUNDARY_CODE"
        public string          BoundaryCodeAttributeTag    { get; set; }

        // AutoCAD block attribute tag that receives the element code value
        // Default: "ELEMENT_ID"
        public string          ElementCodeAttributeTag     { get; set; }

        // Separator inserted between parent code name and the integer suffix
        // Default: "_"  (e.g. "ZONE_A" → "ZONE_A_001")
        public string          AutoGenSuffixSeparator      { get; set; }

        // Zero-padded width of the auto-generated integer suffix
        // Default: 3  (gives "001", "002", …)
        public int             AutoGenSuffixPadding        { get; set; }
    }
}
```

`Services/LiteDBService/LiteDBService.ProjectPluginSettings.cs` ← **NEW**

Purpose: Partial class providing CRUD for `ProjectPluginSettings`. **Dependencies:** `LiteDB`

csharp

```
public partial class LiteDBService
{
    private readonly string projectPluginSettingsCollectionName = "ProjectPluginSettings

    private void init_ProjectPluginSettingsColn();

    // Creates or replaces the settings for a project.
    // Enforces unique index on ProjectObjectId.
    public ProjectPluginSettings Upsert_ProjectPluginSettings(
        ProjectPluginSettings settings);

    // Returns null if no settings exist for this project yet.
    public ProjectPluginSettings GetByProject_ProjectPluginSettings(
        string projectObjectId);

    // Returns a new instance with all defaults pre-populated
    public static ProjectPluginSettings CreateDefaults(string projectObjectId);
}
```

`Services/LiteDBService/model.BlockActivityCodeAssignment.cs` ← **NEW**

Purpose: Tracks which P6 activity codes have been assigned to a specific AutoCAD block reference, identified by its unique handle and the DWG file path. **Dependencies:** `LiteDB`, `PropertyChanged.Fody`

csharp

```
namespace Autocad_Primavera_P6_Plugin.Services.LiteDBService
{
    [AddINotifyPropertyChangedInterface]
    public class BlockActivityCodeAssignment
```

```
{
    [BsonId] public int    Id                       { get; set; }

    // AutoCAD BlockReference.Handle.ToString() — globally unique per DWG
    public string          BlockHandle              { get; set; }

    // Absolute path to the DWG file containing this block
    public string          DWGFilePath              { get; set; }

    // P6 project this block belongs to (from ProjectConfig)
    public string          ProjectObjectId          { get; set; }

    // ── Boundary code assignment ──────────────────────────────
    public int?            BoundaryCodeTypeObjectId { get; set; }
    public int?            BoundaryCodeObjectId     { get; set; }
    public string          BoundaryCodeValue        { get; set; }  // e.g. "ZONE_A"
    public string          BoundaryCodePath         { get; set; }  // e.g. "Root / Z

    // ── Element code assignment ───────────────────────────────
    public int?            ElementCodeTypeObjectId  { get; set; }
    public int?            ElementCodeObjectId      { get; set; }
    public string          ElementCodeValue         { get; set; }  // e.g. "E001_003
    public string          ElementCodePath          { get; set; }  // e.g. "E001 / E

        public DateTime        CreatedAt                { get; set; }
        public DateTime        UpdatedAt                { get; set; }
    }
}
```

`Services/LiteDBService/LiteDBService.BlockAssignments.cs` ← **NEW**

Purpose: Partial class providing CRUD for `BlockActivityCodeAssignment`. **Dependencies:** `LiteDB`

csharp

```
public partial class LiteDBService
{
    private readonly string blockAssignmentsCollectionName = "BlockActivityCodeAssignmen

    private void init_BlockAssignmentsColn();

    // Creates a new assignment or replaces the one with the same
    // (BlockHandle + DWGFilePath) composite key.
    public BlockActivityCodeAssignment Upsert_BlockAssignment(
        BlockActivityCodeAssignment assignment);

    // Finds the assignment for a specific block handle in a specific DWG.
    // Returns null if not yet assigned.
    public BlockActivityCodeAssignment Find_ByHandleAndDWG(
        string blockHandle,
        string dwgFilePath);

    // Returns all assignments that belong to a given project.
    public List<BlockActivityCodeAssignment> GetAll_ByProject(
        string projectObjectId);

    // Deletes by primary key.
    public bool Delete_BlockAssignment(int id);
}
```

`Services/P6ApiService/P6ActivityCodeService.cs` ← **NEW**

Purpose: Wraps the generated `Client` to provide typed, domain-specific operations for activity code types and activity codes. Handles the auto-generate logic (finds next integer-suffixed sibling or child name). Stateless — depends on `P6ApiService.Client` being authenticated. **Dependencies:** `P6ApiService`, generated `Client`, `System.Threading.Tasks`

csharp

```
namespace Autocad_Primavera_P6_Plugin.Services.P6ApiService
{
    public class P6ActivityCodeService
    {
        // ── Constructor ──────────────────────────────────────────
        public P6ActivityCodeService(P6ApiService p6ApiService);

        // ── ActivityCodeType queries ─────────────────────────────

        // Returns all ActivityCodeTypes visible to a given project
        // (scope = Global OR scope = Project with matching ProjectObjectId).
        public Task<List<ActivityCodeType>> GetCodeTypesForProjectAsync(
            int projectObjectId);

        // ── ActivityCode queries ─────────────────────────────────

        // Returns all ActivityCodes that belong to a specific type,
        // flattened (full hierarchy). Use BuildTree() to reconstruct the tree.
        public Task<List<ActivityCode>> GetCodesForTypeAsync(
            int codeTypeObjectId);
```

```
    // Convenience: returns root-level codes for a type (ParentObjectId == 0 or null
    public Task<List<ActivityCode>> GetRootCodesForTypeAsync(
        int codeTypeObjectId);

    // Returns the direct children of a given parent code.
    public Task<List<ActivityCode>> GetChildCodesAsync(
        int parentObjectId,
        int codeTypeObjectId);

    // ── ActivityCode mutations ──────────────────────────────────────

    // Creates a new ActivityCode in P6. Returns the created entity
    // with ObjectId populated by the server.
    // Throws P6ActivityCodeException on failure.
    public Task<ActivityCode> CreateActivityCodeAsync(
        ActivityCode newCode);

    // ── Auto-generate logic ─────────────────────────────────────────

    // Given a parent code (or null for root-level), examines existing siblings
    // and computes the next integer-suffixed name.
    // E.g. siblings ["ZONE_A_001", "ZONE_A_002"] → returns "ZONE_A_003"
    // separator and padding come from ProjectPluginSettings.
    // Does NOT create the code in P6 — call CreateActivityCodeAsync with result.
    public string ComputeNextAutoGeneratedName(
        string parentCodeName,
        IEnumerable<ActivityCode> siblings,
        string separator,
        int padding);

    // ── Tree helper ─────────────────────────────────────────────────

    // Builds a flat list into a proper parent-child tree of ActivityCodes.
    // Returns only the root nodes; children are populated recursively.
    public List<ActivityCode> BuildTree(List<ActivityCode> flat);
}

// Typed exception for P6 activity code API failures
public class P6ActivityCodeException : Exception
{
    public P6ActivityCodeException(string message);
    public P6ActivityCodeException(string message, Exception inner);
}
```

`UserInterface/UI_InsertBlocksWindowView/InsertBlocksWindowModel.cs` ← **UPDATE**

Purpose: Model carries the current selected state for both code sections and block type selection. **Dependencies:** `LiteDBService` models, `P6ApiService` models

csharp

```
public class InsertBlocksWindowModel
{
    // Block type
    public ObservableCollection<string>   AvailableBlocks      { get; set; }

    // Boundary code currently selected (null until user picks)
    public ActivityCode                   SelectedBoundaryCode { get; set; }
    public ActivityCodeType               BoundaryCodeType     { get; set; }

    // Element code currently selected (null until user picks or auto-generates)
    public ActivityCode                   SelectedElementCode  { get; set; }
    public ActivityCodeType               ElementCodeType      { get; set; }

    // The resolved ProjectConfig and settings for the current DWG
    public ProjectConfig                  ActiveProjectConfig  { get; set; }
    public ProjectPluginSettings          ActivePluginSettings { get; set; }
}
```

`UserInterface/UI_InsertBlocksWindowView/InsertBlocksWindowViewModel.cs` ← **UPDATE**

Purpose: Full MVVM orchestration for the Insert Blocks window. Loads project context, manages both code selectors, drives the AutoCAD block insertion. **Dependencies:** `InsertBlocksWindowModel`, `P6ActivityCodeService`, `LiteDBService`, `AutoCAD Application`, `PropertyChanged.Fody`

csharp

```
[AddINotifyPropertyChangedInterface]
public class InsertBlocksWindowViewModel
{
    // ── Display properties for BOUNDARY code section ──────────────
    public string  BoundaryCodeValue     { get; private set; }
    public string  BoundaryCodePath      { get; private set; }
    public string  BoundaryCodeValueId   { get; private set; }
    public bool    IsAutoGenerateBoundary { get; set; }   // not applicable — hidden

    // ── Display properties for ELEMENT code section ───────────────
    public string  ElementCodeValue      { get; private set; }
    public string  ElementCodePath       { get; private set; }
    public string  ElementCodeValueId    { get; private set; }
```

```
    public bool    IsAutoGenerateElement { get; set; }

    // ── Block type section ──────────────────────────────────────
    public bool    IsPredefinedBlockMode { get; set; }
    public ObservableCollection<string>  AvailableBlocks { get; }
    public string  SelectedPredefinedBlock { get; set; }

    // ── Insert options ──────────────────────────────────────────
    public bool    IsContinuousInsert    { get; set; }
    public bool    IsEditLegendPosition  { get; set; }

    // ── Status ──────────────────────────────────────────────────
    public string  StatusMessage         { get; private set; }
    public bool    IsLoading             { get; private set; }
    public bool    IsProjectLinked       { get; private set; }
    public bool    CanInsert             { get; private set; }

    // ── Commands ────────────────────────────────────────────────
    public ICommand SelectBoundaryCodeCommand     { get; }
    public ICommand SelectElementCodeCommand      { get; }
    public ICommand AutoGenerateElementCodeCommand{ get; }
    public ICommand SelectBlockFromDrawingCommand { get; }
    public ICommand InsertBlockCommand            { get; }
    public ICommand CancelCommand                 { get; }
}
```

`UserInterface/UI_InsertBlocksWindowView/InsertBlocksWindowView.xaml` **← UPDATE**

Purpose: Complete XAML replacing all static `Label` placeholders with live data-bound controls and wiring up the six commands. **Dependencies:** ViewModel above, no new XAML namespaces

Key binding points (structure, not full XAML):

```
BOUNDARY ACTIVITY CODE section:
  Code Value  → {Binding BoundaryCodeValue}
  Code Path   → {Binding BoundaryCodePath}
  Code Value Id → {Binding BoundaryCodeValueId}
  Select Button → {Binding SelectBoundaryCodeCommand}
  AutoGenerate CheckBox → hidden (boundary codes are P6-managed only)

ITEM ID ACTIVITY CODE section:
  Code Value  → {Binding ElementCodeValue}
  Code Path   → {Binding ElementCodePath}
  Code Value Id → {Binding ElementCodeValueId}
  Select Button → {Binding SelectElementCodeCommand}
  AutoGenerate CheckBox → {Binding IsAutoGenerateElement}
  AutoGenerate Button   → {Binding AutoGenerateElementCodeCommand}
                          IsEnabled="{Binding IsAutoGenerateElement}"

BLOCK TYPE section:
  Predefined radio → {Binding IsPredefinedBlockMode}
  CopyBlock radio  → {Binding IsPredefinedBlockMode, Converter=InvertBool}
  ComboBox         → {Binding AvailableBlocks} / {Binding SelectedPredefinedBlock}
  Select from drawing → {Binding SelectBlockFromDrawingCommand}

INSERT OPTIONS section:
  Insert Block Button  → {Binding InsertBlockCommand}
                         IsEnabled="{Binding CanInsert}"
  Continuous CheckBox  → {Binding IsContinuousInsert}
  Legend CheckBox      → {Binding IsEditLegendPosition}
  Cancel Button        → {Binding CancelCommand}

Status bar:
  TextBlock → {Binding StatusMessage}
  ProgressBar visibility → {Binding IsLoading, Converter=BoolToVis}
  "No linked project" warning → {Binding IsProjectLinked, Converter=InvertBoolToVis}
```

`UserInterface/UI_UserControls/UI_BoundaryCodeSelector/BoundaryCodeNodeViewModel.cs` **← NEW**

Purpose: Single node in the Boundary Code tree — wraps either an `ActivityCodeType` (root group) or an `ActivityCode` (leaf/branch). **Dependencies:** `P6ApiService` models, `PropertyChanged.Fody`

csharp

```
[AddINotifyPropertyChangedInterface]
public class BoundaryCodeNodeViewModel
{
    // Exactly one of these is non-null
    public ActivityCodeType                  CodeType     { get; set; }
    public ActivityCode                      Code         { get; set; }

    public bool    IsCodeType  => CodeType != null;
    public bool    IsCode      => Code     != null;

    // Display
    public string  Id          { get; }   // CodeType.Name or Code.Name
    public string  Name        { get; }   // same — boundary codes show only Name
    public string  FullPath    { get; }   // computed: "TypeName / ParentName / Name"
```

```
    // Tree state (bound to TreeViewItem)
    public bool    IsSelected  { get; set; }
    public bool    IsExpanded  { get; set; }
    public bool    IsVisible   { get; set; }

    public ObservableCollection<BoundaryCodeNodeViewModel> Children { get; }

    // Fires the ViewModel's selection-changed callback
    private readonly Action<BoundaryCodeNodeViewModel> _onSelectedCallback;
    public void OnIsSelectedChanged();   // Fody callback
}
```

`UserInterface/UI_UserControls/UI_BoundaryCodeSelector/BoundaryCodeSelectorViewModel.cs`
← NEW

Purpose: Loads the P6 activity code type + code hierarchy for boundary codes, provides search/filter, exposes the selected code for the caller. **Dependencies:** `P6ActivityCodeService`, `BoundaryCodeNodeViewModel`, `PropertyChanged.Fody`

csharp

```
[AddINotifyPropertyChangedInterface]
public sealed class BoundaryCodeSelectorViewModel
{
    // ── State ──────────────────────────────────────────────────────────
    public ObservableCollection<ActivityCodeType>       AvailableCodeTypes { get; }
    public ActivityCodeType                             SelectedCodeType   { get; set; }
    public ObservableCollection<BoundaryCodeNodeViewModel> RootNodes       { get; }
    public BoundaryCodeNodeViewModel                    SelectedNode       { get; set; }

    public bool    IsCodeSelected  => SelectedNode?.IsCode == true;
    public ActivityCode            SelectedCode   => IsCodeSelected ? SelectedNode.Code

    public bool    IsLoading       { get; private set; }
    public string  StatusMessage   { get; private set; }
    public string  SearchString    { get; set; }
    public bool    IsCancelled     { get; private set; }

    public event Action<bool?> RequestClose;

    // ── Commands ───────────────────────────────────────────────────────
    public ICommand LoadCodeTypeCommand  { get; }   // fires when SelectedCodeType chang
    public ICommand SearchCommand        { get; }
    public ICommand OkCommand            { get; }
    public ICommand CancelCommand        { get; }

    // ── Constructor ────────────────────────────────────────────────────
    public BoundaryCodeSelectorViewModel(
        MyPlugin pluginInstance,
        string   projectObjectId);

    // ── Public async entry ─────────────────────────────────────────────
    public Task LoadAsync();

    // ── Private methods (signatures only) ──────────────────────────────
    private Task   LoadCodeTypesAsync();
    private Task   LoadCodesForSelectedTypeAsync();
    private void   BuildCodeTree(List<ActivityCode> flat);
    private bool   FilterNode(BoundaryCodeNodeViewModel node, string filter);
    private void   ResetVisibility(BoundaryCodeNodeViewModel node);
    private void   NodeSelectedHandler(BoundaryCodeNodeViewModel node);
    private void   OnSelectedCodeTypeChanged();   // Fody — triggers LoadCodesForSelecte
}
```

`UserInterface/UI_UserControls/UI_BoundaryCodeSelector/BoundaryCodeSelectorView.xaml`
← NEW

Purpose: Modal dialog window with a code-type ComboBox at the top and a searchable TreeView of codes below. OK is enabled only when `IsCodeSelected == true`.

Key layout regions:

```
Row 0: Header ("Select Boundary Activity Code" + subtitle)
Row 1: CodeType ComboBox (bound to AvailableCodeTypes / SelectedCodeType)
Row 2: Search TextBox + Search Button
Row 3: TreeView (bound to RootNodes, same ItemContainerStyle as P6ProjectSelectorView)
       Side panel: Ok Button (IsEnabled={Binding IsCodeSelected})
                   Cancel Button
Row 4: Status bar (IsLoading indicator + StatusMessage)
```

`UserInterface/UI_UserControls/UI_BoundaryCodeSelector/BoundaryCodeSelectorView.xaml.cs`
← NEW

Purpose: Code-behind. Wires `RequestClose` event to `DialogResult` + `Close()`.

csharp

```
public partial class BoundaryCodeSelectorView : Window
{
    public BoundaryCodeSelectorView(BoundaryCodeSelectorViewModel viewModel);
```

```
    private void OnRequestClose(bool? dialogResult);
}
```

`UserInterface/UI_UserControls/UI_ElementCodeSelector/ElementCodeNodeViewModel.cs` ← **NEW**

Purpose: Identical structure to `BoundaryCodeNodeViewModel` plus one extra property indicating whether this node can have auto-generated children. **Dependencies:** same as `BoundaryCodeNodeViewModel`

csharp

```
[AddINotifyPropertyChangedInterface]
public class ElementCodeNodeViewModel
{
    public ActivityCodeType                  CodeType          { get; set; }
    public ActivityCode                      Code              { get; set; }

    public bool    IsCodeType     => CodeType != null;
    public bool    IsCode         => Code     != null;

    // True when this node is an ActivityCode (not a type header)
    // and the user can place a new child under it via auto-generate.
    public bool    CanAutoGenerateChild { get; }

    public string  Id             { get; }
    public string  Name           { get; }
    public string  FullPath       { get; }

    public bool    IsSelected     { get; set; }
    public bool    IsExpanded     { get; set; }
    public bool    IsVisible      { get; set; }

    public ObservableCollection<ElementCodeNodeViewModel> Children { get; }

    private readonly Action<ElementCodeNodeViewModel> _onSelectedCallback;
    public void OnIsSelectedChanged();
}
```

`UserInterface/UI_UserControls/UI_ElementCodeSelector/ElementCodeSelectorViewModel.cs` ← **NEW**

Purpose: Extends boundary-code selector concept with auto-generate mode. When `IsAutoGenerateMode == true`, the OK button is replaced by "Create & Assign" which calls `P6ActivityCodeService.ComputeNextAutoGeneratedName` → `CreateActivityCodeAsync` then closes with the new code. **Dependencies:** `P6ActivityCodeService`, `LiteDBService.ProjectPluginSettings`, `ElementCodeNodeViewModel`, `PropertyChanged.Fody`

csharp

```
[AddINotifyPropertyChangedInterface]
public sealed class ElementCodeSelectorViewModel
{
    // ── All properties from BoundaryCodeSelectorViewModel, plus: ──────────
    public ObservableCollection<ActivityCodeType>        AvailableCodeTypes  { get; }
    public ActivityCodeType                              SelectedCodeType    { get; set;
    public ObservableCollection<ElementCodeNodeViewModel> RootNodes          { get; }
    public ElementCodeNodeViewModel                      SelectedNode        { get; set;

    public bool    IsCodeSelected => SelectedNode?.IsCode == true;
    public ActivityCode            SelectedCode  => IsCodeSelected ? SelectedNode.Code :

    // ── Auto-generate specific ─────────────────────────────────────────────
    public bool    IsAutoGenerateMode    { get; set; }
    public string  ProposedNewCodeName   { get; private set; }  // preview, e.g. "ZONE_A
    public string  ProposedNewCodePath   { get; private set; }  // e.g. "ZONE_A / ZONE_A
    public bool    IsCreatingCode        { get; private set; }  // spinner during P6 cal

    // ── Computed CanXxx ────────────────────────────────────────────────────
    // True when NOT in auto-generate mode and a code node is selected
    public bool    CanSelectExisting     { get; }
    // True when in auto-generate mode and a valid parent (or root) is chosen
    public bool    CanCreateAndAssign    { get; }

    public bool    IsLoading             { get; private set; }
    public string  StatusMessage         { get; private set; }
    public string  SearchString          { get; set; }
    public bool    IsCancelled           { get; private set; }

    public event Action<bool?> RequestClose;

    // ── Commands ──────────────────────────────────────────────────────────
    public ICommand LoadCodeTypeCommand      { get; }
    public ICommand SearchCommand            { get; }
    public ICommand OkCommand                { get; }   // select existing
    public ICommand CreateAndAssignCommand   { get; }   // auto-generate + create in P6
    public ICommand CancelCommand            { get; }

    // ── Constructor ───────────────────────────────────────────────────────
    public ElementCodeSelectorViewModel(
        MyPlugin              pluginInstance,
        string                projectObjectId,
```

```
        bool                  startInAutoGenerateMode = false);

    // ── Private method signatures ──────────────────────────────────────
    private Task   LoadCodeTypesAsync();
    private Task   LoadCodesForSelectedTypeAsync();
    private void   BuildCodeTree(List<ActivityCode> flat);
    private void   RefreshProposedName();       // recalculates on SelectedNode change
    private bool   FilterNode(ElementCodeNodeViewModel node, string filter);
    private void   ResetVisibility(ElementCodeNodeViewModel node);
    private void   NodeSelectedHandler(ElementCodeNodeViewModel node);
    private Task   ExecuteCreateAndAssignAsync();
    private void   OnSelectedNodeChanged();     // Fody → RefreshProposedName
    private void   OnIsAutoGenerateModeChanged();
    private void   OnSelectedCodeTypeChanged();
}
```

`UserInterface/UI_UserControls/UI_ElementCodeSelector/ElementCodeSelectorView.xaml` ←
NEW

Purpose: Modal window with code-type ComboBox, search row, TreeView, a toggle for auto-generate mode, and a preview panel that shows the proposed new code name when in auto-generate mode.

Key layout regions:

```
Row 0: Header ("Select Element Activity Code")
Row 1: CodeType ComboBox
Row 2: Search bar
Row 3: LEFT: TreeView (RootNodes)
       RIGHT: Panel with:
              [Mode toggle CheckBox: "Auto-generate new child code"]
              [Proposed name preview box (Visibility=IsAutoGenerateMode)]
                - "New code: {ProposedNewCodeName}"
                - "Under:    {ProposedNewCodePath}"
              [Ok Button        IsEnabled=CanSelectExisting
                                Visibility=NOT IsAutoGenerateMode]
              [Create & Assign  IsEnabled=CanCreateAndAssign
                                Visibility=IsAutoGenerateMode]
              [Cancel Button]
Row 4: Status bar
```

`UserInterface/UI_UserControls/UI_ElementCodeSelector/ElementCodeSelectorView.xaml.cs`
← NEW

Purpose: Code-behind; identical pattern to `BoundaryCodeSelectorView.xaml.cs`.

csharp

```
public partial class ElementCodeSelectorView : Window
{
    public ElementCodeSelectorView(ElementCodeSelectorViewModel viewModel);
    private void OnRequestClose(bool? dialogResult);
}
```

Data flow summary (for Phase 2 reference)

```
User opens Insert Blocks Window
  └─ ViewModel.ctor
       ├─ LiteDBService.Find_byAcadDWG(doc) → ProjectConfig
       ├─ LiteDBService.GetByProject_ProjectPluginSettings(projectObjectId) →
ProjectPluginSettings
       └─ IsProjectLinked = (ProjectConfig != null)

User clicks "Select" (Boundary)
  └─ BoundaryCodeSelectorViewModel.LoadAsync()
       ├─ P6ActivityCodeService.GetCodeTypesForProjectAsync(projectObjectId)
       ├─ [user picks code type from ComboBox]
       ├─ P6ActivityCodeService.GetCodesForTypeAsync(codeTypeObjectId)
       └─ [user selects a leaf node, clicks Ok]
  └─ InsertBlocksWindowViewModel receives ActivityCode
       └─ populates BoundaryCodeValue / Path / ValueId

User clicks "Select" or "Auto-generate" (Element)
  └─ ElementCodeSelectorViewModel.LoadAsync()
       ├─ same code-type loading...
       ├─ IsAutoGenerateMode = true (if auto-gen was clicked)
       ├─ [user selects parent node]
       ├─ RefreshProposedName() → ComputeNextAutoGeneratedName(siblings, sep, pad)
       └─ [user clicks "Create & Assign"]
            ├─ P6ActivityCodeService.Cre                yCodeAsync(newCode)
            └─ RequestClose(true) with the newly created ActivityCode

User clicks "Insert Block"
  └─ InsertBlocksWindowViewModel.InsertBlock()
       ├─ Validates: both codes selected, block type chosen
       ├─ Opens AutoCAD transaction
       │    ├─ BlockReference insert at user-picked point
       │    ├─ AttributeReference write: tag=ElementCodeAttributeTag →
ElementCodeValue
```

```
       |    ├─ AttributeReference write: tag=BoundaryCodeAttributeTag → BoundaryCodeValue
       |    ├─ tr.Commit()
       |    └─ LiteDBService.Upsert_BlockAssignment(assignment with handle+DWG+codes)
       └─ if IsContinuousInsert → loop back to pick-point prompt
```

Phase 1 is complete. All 14 new/updated files have defined paths, stated purposes, external dependencies, and full public signatures. No implementation details are included — that is Phase 2.

Please confirm when you are ready for **Phase 2: Implementation Blueprint & Step-by-Step Pseudo-code**.